Wiederholungsklausur Informatik I
Sommersemester 2011
Prof. Dr. C. Damm
08. April 2011
von 08:00+x bis 09:30+x Uhr

Vorname:

Nachname:

Matrikelnummer:

Es sind **keine Hilfsmittel** (Skript, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Schreiben Sie mit blauem/schwarzem Kugelschreiber, Füller etc.. Bleistift ist nicht erlaubt. Unlesbare Antworten werden nicht gewertet. Algorithmen sind in Java-Code zu notieren, sofern keine Alternativen genannt sind.

Viel Erfolg!

	Max. Punkte	Erreichte Punkte
Aufgabe 1 (Auswahlaufgaben)	15	
Aufgabe 2 (Elementare und strukturierte Daten)	15	
Aufgabe 3 (Allgemeine Kenntnisfragen)	15	
Aufgabe 4 (Rekursion und Iteration)	15	
Aufgabe 5 (Algorithmen-Entwurf und Analyse)	15+2	
Aufgabe 6 (Vererbung und dynamische Bindung)	15	
Summe	90+2	

Note:

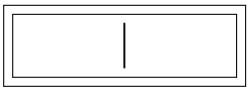
Name: MatNr.:

Aufgabe 1 (Auswahlaufgaben [15 Punkte])

Jeweils nur eine Antwort ankreuzen (**X**). Mehrere Kreuze ergeben 0P für die Teilaufgabe.

- `class A {void tuWas(){..}.. {void tuWas(int x){..}}`
  Was ist richtig?
  1. `tuWas` wurde erweitert.
  2. `tuWas` wurde überladen.
  3. `tuWas` wurde überschrieben.
- Klassen ...
  1. ... sind Mengen von Objekten.
  2. ... sind ausführbare Quellcodeeinheiten.
  3. ... geben die Struktur von Objekten vor.
- `a && !b` ist gleichbedeutend mit ...
  1. weder `a` noch `b`
  2. aus `a` folgt `b`
  3. es gilt nicht: aus `a` folgt `b`
- Eine Schleifeninvariante ist ...
  1. ... eine Aussage.
  2. ... eine Spezifikation.
  3. ... immer eindeutig bestimmt.
- Der Garbage-Collector ...
  1. ... entfernt fehlerhaften Code.
  2. ... gibt Speicherplatz frei.
  3. ... läuft parallel zum Compiler.
- Ein vollständiger binärer Baum ...
  1. ... hat noch alle Blätter.
  2. hat eine Wurzel.
  3. ... hat Tiefe 2^k.
- Mergesort ist ...
  1. ... ein in-place-Verfahren.
  2. ... dynamisch programmiert.
  3. ... asymptotisch Laufzeit-optimal.
- Die von-Neumann-Architektur ...
  1. ... sieht Programm und Daten im gleichen Speicher vor.
  2. ... ermöglicht Programmierung ohne bedingte Sprünge.
  3. ... liegt dem EVA-Prinzip zugrunde.

- Der Aktivierungssatz ...
  1. ... beschreibt die Struktur des Laufzeitstapels.
  2. ... wurde von Leibniz bewiesen.
  3. ... hat für jeden Aufruf einer Methode die gleiche Struktur.
- Durch Stellenwertsysteme ...
  1. ... wird Rechnen mit Zahlen auf das Rechnen mit Ziffern zurückgeführt.
  2. ... ist die Syntax jeder Stelle vorgeschrieben.
  3. ... sind auch nicht druckbare Zeichen im Unicode.
- Aufrufe von funktionalen Methoden ...
  1. ... können überall da stehen, wo ein Ausdruck erlaubt ist.
  2. ... dürfen nicht `null` liefern
  3. ... sind nur außerhalb von Konstruktoren erlaubt.
- Eine Queue ...
  1. ... ist eine doppelt verkettete Liste.
  2. ... hat im schlechtesten Fall logarithmische Zugriffszeit.
  3. ... verwirklicht auch das Prinzip last in, last out.
- Mit `final` kann man ...
  1. ... Klassen ohne Konstruktor kennzeichnen.
  2. ... symbolische Konstanten definieren.
  3. ... nur Methoden markieren, die `return` enthalten.
- Binäre Suche ...
  1. ... benötigt auf Feldern der Länge n mindestens $\mathcal{O}(n \log n)$ Schritte.
  2. ... ist das schnellste Sortierverfahren.
  3. ... ist nur unter bestimmten Bedingungen auf Felder anwendbar.
- Compilezeitfehler sind Fehler ...
  1. ... die die Compilezeit verschieben.
  2. ... die den Compiler zum Absturz bringen.
  3. ... in einem Quelltext.



Name: MatNr.:

Aufgabe 2 (Elementare und strukturierte Daten [15 Punkte])

a) (3P) Der hypothetische Datentyp `gint` stellt ganze Zahlen mit 6-Bit-Zweierkomplement dar. Geben Sie folgende Werte dezimal an:

- größter darstellbarer `gint`-Wert
- kleinster darstellbarer `gint`-Wert.

b) (3P) Wandeln Sie den Dezimalbruch $(0.203125)_{10}$ in einen Binärbruch um.

c) (6P) Ergänzen Sie diese Tabelle von `byte`-Werten (= „8-Bit-Ganzzahlen“).

Bitte deutlich schreiben, Nebenrechnung nur unten oder auf Blatt rechts!

dezimal	binär (mit Zweierkompl.)	hexadezimal
13		
	$(00101011)_{\overline{2}}$	
		$(1E)_{16}$

d) (3P) Betrachten Sie folgende Definition:

```
int[] f = new int[13];
```

Die Anfangsadresse von f sei `0x1200A011`. Welche Adresse hat `f[7]`? Geben Sie den Rechenweg an.

Name: MatNr.:

Aufgabe 3 (Allgemeine Kenntnisfragen [15 Punkte])
3P pro Aufgabenteil. Antworten Sie kurz und prägnant.

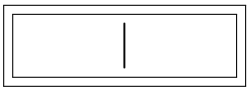
1. `static int bla(int x){int z = 21; return x+1;}` wird in `main` mit `int z = bla(7);` aufgerufen. Was wird im Aktivierungssatz gespeichert?

2. Welche Elemente einer Java-Klassendefinition werden durch das Schlüsselwort `static` gekennzeichnet? Was bedeutet `private`?

3. Nennen Sie drei Funktionseinheiten der von Neumann-Rechnerarchitektur.

4. Wodurch ist imperative Programmierung gekennzeichnet?

5. Die Tiefe eines Baums ist die maximale Kantenzahl längs eines Wurzel↦Blatt-Weges. Wieviele Knoten hat ein Tiefe-n-Binärbaum mindestens, wieviel höchstens?



Name: MatNr.:

Aufgabe 4 (Rekursion und Iteration [15 Punkte])

1.(5P) Die Java-Methode `static String rev(String s)` liefert den „Umkehrstring“ von `s` (enthält die Zeichen von `s` in umgekehrter Reihenfolge, Beispiel `rev("Maus")` liefert `"suaM"`).
Programmieren Sie die Methode rekursiv mit der Rekursionsidee

$$\texttt{rev}(\texttt{s}) = \begin{cases} \texttt{s} & \text{, falls} |\texttt{s}| \leq 1 \\ \texttt{rev}(\texttt{s}_{1..n-1}) + \texttt{s}_0 & \text{sonst.} \end{cases}$$

Dabei stehen $|\texttt{s}|$ für die Länge und $\texttt{s}_0$ für das Anfangszeichen von `s`, $\texttt{s}_{1..n-1}$ für den String ohne Anfangszeichen und + für die Verkettung (z.B. $ab{+}a = aba$).
- Formulieren Sie nur die Methode, keine Klasse!
- Benutzen Sie `length`, `charAt`, `substring` (s.u.) und den Operator `+`:

```
// Laenge von s = Anzahl der Zeichen darin (Bsp. "Maus" hat Länge 4)
int len = s.length();

// erstes Zeichen von s:
char c = s.charAt(0);

// alle Zeichen von s vom einschliesslich zweiten bis letzten:
String rest = s.substring(1);
```

2. (5P) Schreiben Sie eine iterative Methode `REV`, die das Gleiche leistet wie `rev`.
- Formulieren Sie nur die Methode, keine Klasse!
- Benutzen Sie `length` und `charAt` (s.o.) und den Operator `+`:

3. (5P) Betrachten Sie folgende Methode

```
static int quasi(int b, int e) {
  int p = 1;
  if (e > 0) {
    p = quasi(b,e/2) - 1;
    p = p*p + 1;
    if ( e % 2 != 0)
        p *= b;
    }
  return p;
}
```

Welchen Wert liefert der Aufruf `quasi(2,5);` ? Die richtige Lösung zählt nur, wenn auch der Rechenweg angegeben wird (sonst 0P).

Name: MatNr.:

Aufgabe 5 (Algorithmen-Entwurf und Analyse [15+2 Punkte])
Gegeben ist eine Zeichenfolge, z.B. "`cwNHOpoQMMNIsQnUaZbAsG`"
gespeichert in einem Feld `z[0]...z[k-1]`.

a) (6P + (2 Zusatzpunkte für asymptotisch optimalen Algorithmus)) Geben Sie einen Algorithmus an, der berechnet, wie lang die längste Teilfolge ist, die nur aus Großbuchstaben besteht (im Beispiel würde 5 berechnet werden, denn die Folge hat keine längere Großbuchstabenfolge als `QMMNI`). Geben Sie den Algorithmus in "strukturierter Prosa"[2] an (kein Java!).

[2] knapp & übersichtlich, Beispiel: Berechnung des Mittelwerts eingegebener Zahlen

1. Initialisierung: Setze Anzahl n und Summe s auf 0.
2. Ende erreicht?: Ist keine Zahl mehr zu lesen, weiter bei 4.
3. Einlesen und Verarbeiten einer Zahl:
   - Lies nächste Eingabezahl z.
   - Erhöhe a um 1 und s um z.
   - Weiter bei 2.
4. Berechnung und Beendigung: Falls $a > 0$, liefere Mittelwert s/a zurück, sonst erzeuge Fehlermeldung

b) (5P) Geben Sie eine möglichst gute Abschätzung Ihres Algorithmus in $\mathcal{O}$-Notation an (mit kurzer Begründung).

c) (4P) Algorithmus `clever` zur Lösung von a) ermittelt zunächst die Feldlänge und liest dann beginnend bei Index 0 fortlaufend die Feldeinträge (und stellt interne Berechnungen beliebiger Art an). Das Lesen wird aber sofort unterbrochen, wenn die Antwort feststeht.

Geben Sie Zeichenketten (1), (2) und (3) der Länge 8 wie folgt an: (1) `clever` liest bis zum Ende. (2) `clever` liest nicht bis zum Ende und das Resultat ist 4. (3) `clever` liest nicht bis zum Ende und das Resultat ist 2.

Begründen Sie jeweils kurz.

Name: MatNr.:

Aufgabe 6 (Vererbung und dynamische Bindung [15 Punkte])

1. (8P) Geben Sie unten die beim Aufruf `java PersonTest` entstehenden Ausgaben an (Nummerierung: siehe Kommentar der Zeilen 7–11).

```
public class PersonTest {
    public static void main(String args[]) {
        Person jemand = new Person("Tom", "Muster", false, "Dr.");
        Heiliger martin = new Heiliger("Martin", "", false);
        Heiliger anna = new Heiliger("Anna", "", true);
        Herrscher kw = new Herrscher("Kalle", "Wirsch", false);
        System.out.println(jemand.bezeichnung()); // [1]
        System.out.println(jemand.anrede()); // [2]
        System.out.println(martin.bezeichnung()); // [3]
        System.out.println(anna.anrede()); // [4]
        System.out.println(kw.anrede()); // [5]
    }
}
class Person {
    private String vorname, nachname, titel;
    private boolean weiblich;
    public Person(String v, String n, boolean w, String t) {
        vorname = v; nachname = n; titel = t; weiblich = w;
    }
    public String vorname() { return vorname; }
    public String nachname() { return nachname; }
    public String titel() { return titel; }
    public boolean weiblich() { return weiblich; }
    public String name() { return vorname + " " + nachname; }
    public String bezeichnung() { return titel() + " " + name(); }
    public String anrede() {
        if (weiblich)
            return "Sehr geehrte Frau " + bezeichnung();
        else
            return "Sehr geehrter Herr " + bezeichnung();
    }
}
class Heiliger extends Person {
    public Heiliger(String v, String n, boolean w) {
        super(v, n, w, "St.");
    }
    public String bezeichnung() {
        if (weiblich())
            return "Heilige " + titel() + " " + vorname();
        else
            return "Heiliger " + titel() + " " + vorname();
    }
}
class Herrscher extends Person {
    public Herrscher(String v, String n, boolean w) {
        super(v, n, w, "");
    }
    public String titel() {return "Hoheit";}
}
```

Ausgabe 1:
Ausgabe 2:
Ausgabe 3:
Ausgabe 4:
Ausgabe 5:

2. (7P) Vervollständigen Sie anhand der Klassen `Angestellter` und `Chef` (im gleichen Verzeichnis) die nachfolgende Tabelle mit den Einträgen "korrekt" oder "Compilezeitfehler" oder "Laufzeitfehler", je nachdem, ob die Anweisung(en) in der linken Spalte unproblematisch sind oder welche Art Fehler verursacht wird.

```
public class Angestellter {
 protected int alter;
 protected String name;
 public Angestellter(String name, int alter) {
  this.name = name;
  this.alter = alter;}
 public String toString() {
  return name + " (" + alter + ")";}
class Chef extends Angestellter {
 protected String abteilung;
 public Chef(String name, int alter, String abteilung) {
  super(name,alter);
  this.abteilung = abteilung;}
 public String toString() {
  String s = super.toString();
  return s + " leitet " + abteilung; }
}
```

Anweisung(en)	Eintrag

`String s, abt;`
`Angestellter a=new Angestellter("Berthold",30);`
`Chef b=new Chef("Steinke",37,"Schadensregulierung");`	

`s = a.name;`	
`abt = a.abteilung;`	
`abt = ((Chef) a).abteilung;`	
`b = a;`	
`a = b;`	
`a = b;` `abt = a.abteilung;`	
`a = b;` `abt = ((Chef) a).abteilung;`	
`System.out.println(((Angestellter) a).toString());`	
`System.out.println(((Angestellter) b).toString());`	

Name: MatNr.: